

```

LM agent
# You may need to add your working directory to the Python path. To do so,
uncomment the following lines of code
# import sys
# sys.path.append("/Path/to/directory/agent-framework") # Replace with
your directory path

import logging

from baf.core.agent import Agent
from baf.core.session import Session
from baf.exceptions.logger import logger
from baf.nlp.intent_classifier.intent_classifier_configuration import
LLMIntentClassifierConfiguration
from baf.nlp.llm.llm_huggingface import LLMHuggingFace
from baf.nlp.llm.llm_huggingface_api import LLMHuggingFaceAPI
from baf.nlp.llm.llm_openai_api import LLMOpenAI
from baf.nlp.llm.llm_replicate_api import LLMReplicate

# Configure the logging module (optional)
logger.setLevel(logging.INFO)

# Create the agent
agent = Agent('llm_agent')
# Load agent properties stored in a dedicated file
agent.load_properties('config.yaml')
# Define the platform your agent will use
websocket_platform = agent.use_websocket_platform(use_ui=True)

# Create the LLM
gpt = LLMOpenAI(
    agent=agent,
    name='gpt-4o-mini',
    parameters={},
    num_previous_messages=10
)

# Other example LLM

# gemma = LLMHuggingFace(agent=agent, name='google/gemma-2b-it',
parameters={'max_new_tokens': 1}, num_previous_messages=10)
# llama = LLMHuggingFaceAPI(agent=agent,
name='meta-llama/Meta-Llama-3.1-8B-Instruct', parameters={},
num_previous_messages=10)
# mixtral = LLMReplicate(agent=agent,
name='mistralai/mixtral-8x7b-instruct-v0.1', parameters={},
num_previous_messages=10)

ic_config = LLMIntentClassifierConfiguration(
    llm_name='gpt-4o-mini',
    parameters={},
    use_intent_descriptions=True,
    use_training_sentences=False,

```

```

        use_entity_descriptions=True,
        use_entity_synonyms=False
    )
agent.set_default_ic_config(ic_config)

# STATES

greetings_state = agent.new_state('greetings_state', initial=True)
answer_state = agent.new_state('answer_state')

# INTENTS

hello_intent = agent.new_intent(
    name='hello_intent',
    description='The user greets you'
)

maths_intent = agent.new_intent(
    name='maths_intent',
    description='The user asks something about mathematics'
)

physics_intent = agent.new_intent(
    name='physics_intent',
    description='The user asks something about physics'
)

literature_intent = agent.new_intent(
    name='literature_intent',
    description='The user asks something about literature'
)

psychology_intent = agent.new_intent(
    name='psychology_intent',
    description='The user asks something about psychology'
)

# STATES BODIES' DEFINITION + TRANSITIONS

def global_fallback_body(session: Session):
    answer = gpt.predict(f"You are being used within an intent-based agent.
The agent triggered the fallback mechanism because no intent was recognized
from the user input. Generate a message similar to 'Sorry, I don't know the
answer', based on the user message: {session.event.message}")
    session.reply(answer)

agent.set_global_fallback_body(global_fallback_body)

def greetings_body(session: Session):

```

```
    answer = gpt.predict(f"You are a helpful assistant. Start the
conversation with a short (2-15 words) greetings message. Make it
original.")
    session.reply(answer)
```

```
greetings_state.set_body(greetings_body)
# Here, we could create a state for each intent, but we keep it simple
greetings_state.when_intent_matched(hello_intent).go_to(greetings_state)
greetings_state.when_intent_matched(maths_intent).go_to(answer_state)
greetings_state.when_intent_matched(physics_intent).go_to(answer_state)
greetings_state.when_intent_matched(literature_intent).go_to(answer_state)
greetings_state.when_intent_matched(psychology_intent).go_to(answer_state)
```

```
def answer_body(session: Session):
    answer = gpt.predict(session.event.message)
    session.reply(answer)
```

```
answer_state.set_body(answer_body)
answer_state.go_to(greetings_state)
```

```
# RUN APPLICATION
```

```
if __name__ == '__main__':
    agent.run()
```